CONNECTING THE DOTS

ISC High Performance

Energy monitoring, management and evaluation

Basic concepts about power, energy, and performance and metrics

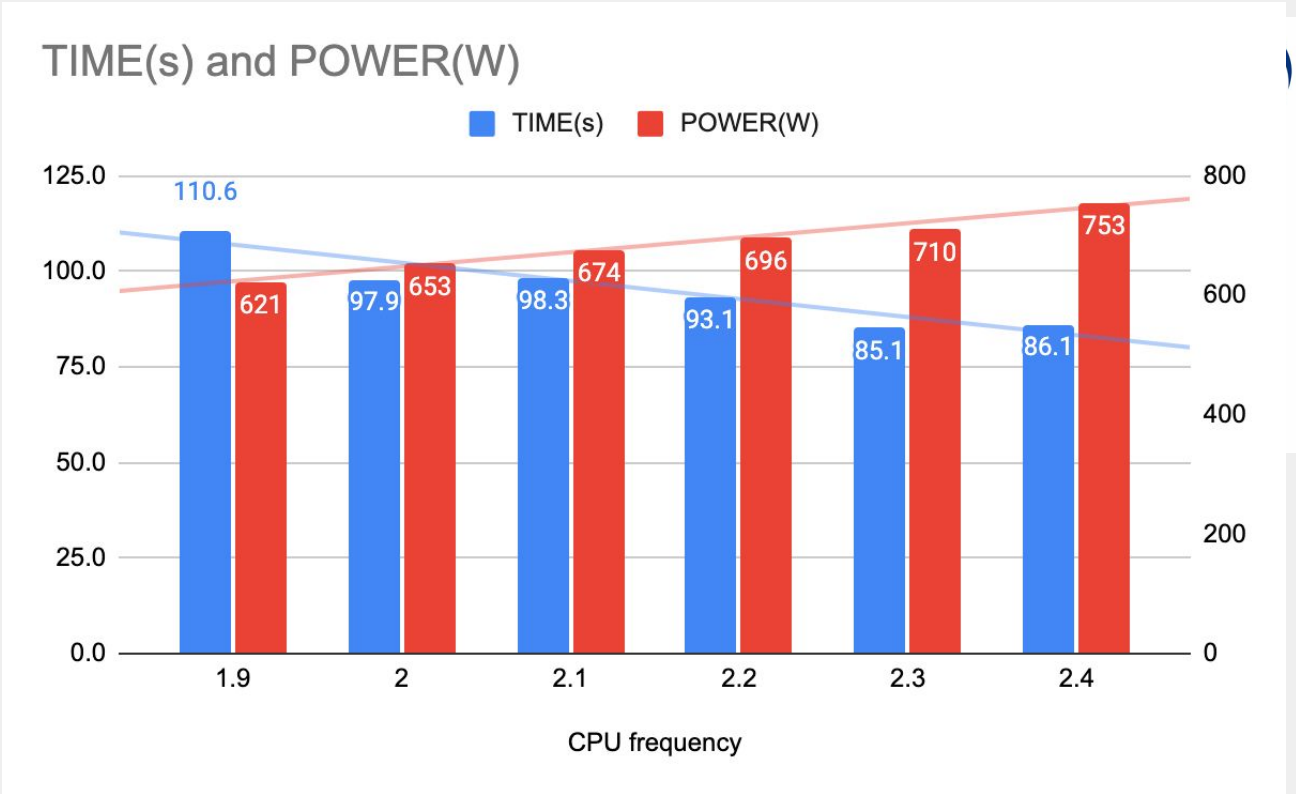
Energy/power monitoring

- Multiple sources of "sensors"
  - Power → instantaneous power
  - Energy → accumulated over the time
- DC node power
  - In band: Through the OS. Typically IPMI, Redfish etc.
  - Contacts the BCM
  - CPU+DRAM+GPU+Others (not constant)
- CPU power
  - Cores+Cache
  - RAPL(through msr, perf, hwmon, etc), HSMP(AMD)
- DRAM power
  - RAPL (through msr, perf, hwmon, etc), HSMP
- GPU
  - NVIDIA: libnvml-ml
  - AMD: rocm-smi
  - Intel: levelzero

Important for fair comparison!

Energy management

- Time and Power varies with CPU frequency
- It depends on the CPU model, CPU settings, and application dynamic characteristics
  - CPU bound shows more linear variation than memory bound applications
- Energy = Time x Power
  - Joules
  - KWH (energy/3600000)



Dynamic optimization

- OS guided
  - cpufreq governors for dynamic cpufreq selection
  - Based on Core “load”
  - Load is not the same as Real utilization , for example, a busy waiting loop shows a very high load but not effective utilization
- Runtime solutions, for example EAR runtime
  - Measures, at runtime, application activity, with more semantic than only load
  - Detects energy savings opportunities
    - Low activity
    - No computational phases
    - No CPU bound phases
    - etc
  - Apply energy optimization policies

Energy evaluation

- Energy = Time x Power (Joules or KW/H)
- Co2
  - CO2 = Energy (KWH) x Carbon Intensity x PUE
    - Carbon intensity factor depends on the country , source of energy etc
    - PUE depends on the DC
- Performance evaluation
  - Time (to solution)
  - Other metrics
    - GFlops
    - Application specific
- We propose to use more than one metric :
  - Energy + Performance + Performance/Watt

Tutorial organization

- Context
  - EAR
  - Snellius
- Energy monitoring, understanding and visualization in Snellius
  - CPU
  - GPU
- Energy optimization & Advance use cases
  - CPU
  - GPU: Singularity, DCGMI metrics, optimization
- Materials
  - Codes in /projects/0/energy-course
  - GIT: https://github.com/sara-nl/ISC-EAR-tutorial/tree/main
  - Get the examples and test them
    - https://github.com/sara-nl/ISC-EAR-tutorial/tree/main/scripts
  -

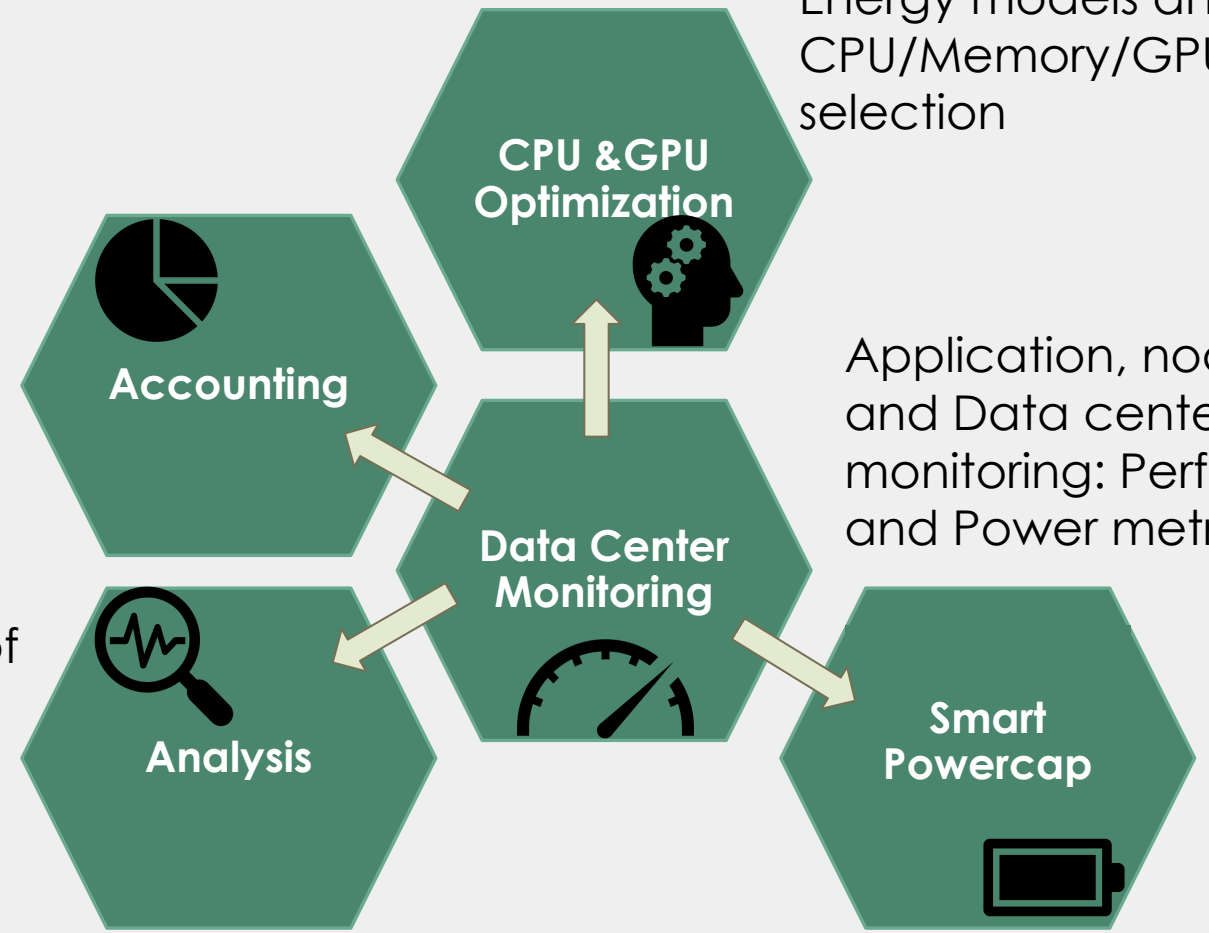
EAR overview

Julita Corbalan (julita.corbalan@eas4dc.com, julita.corbalan@bsc.es)
Benjamin Czaja (benjamin.czaja@surf.nl)
Martijn Kruiter (martijn.kruiten@surf.nl)

EAR features

Energy models and policies for CPU/Memory/GPU frequency selection

Data reporting for accounting, billing, visualization and analytics



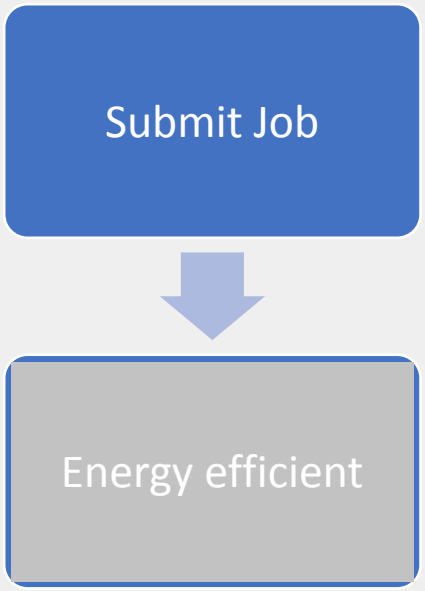
Application, node , cluster and Data center monitoring: Performance and Power metrics

Analysis and classification of application metrics for energy and power optimization

Power control to guarantee data center operational limits

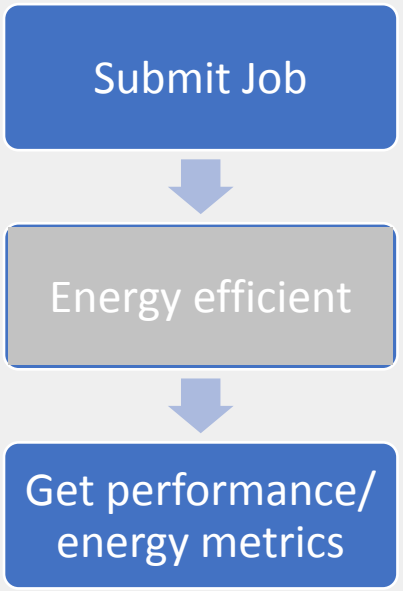
What EAR can do for you?

0 effort



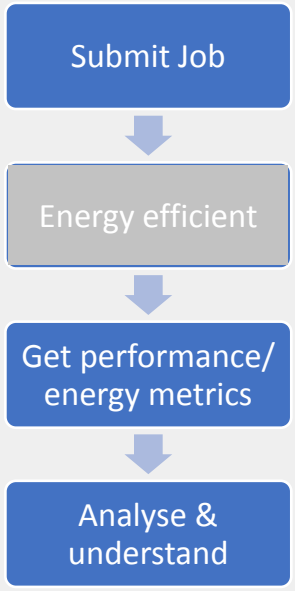
Evaluate my app

I want to know



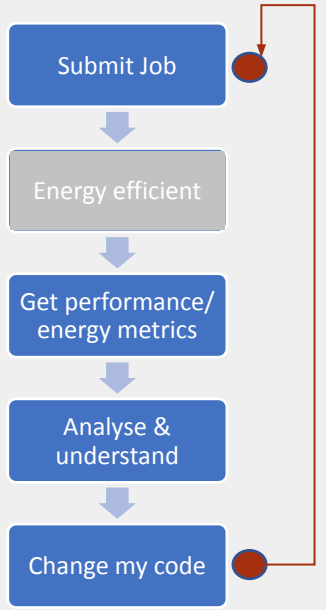
Understand my app

I want to understand



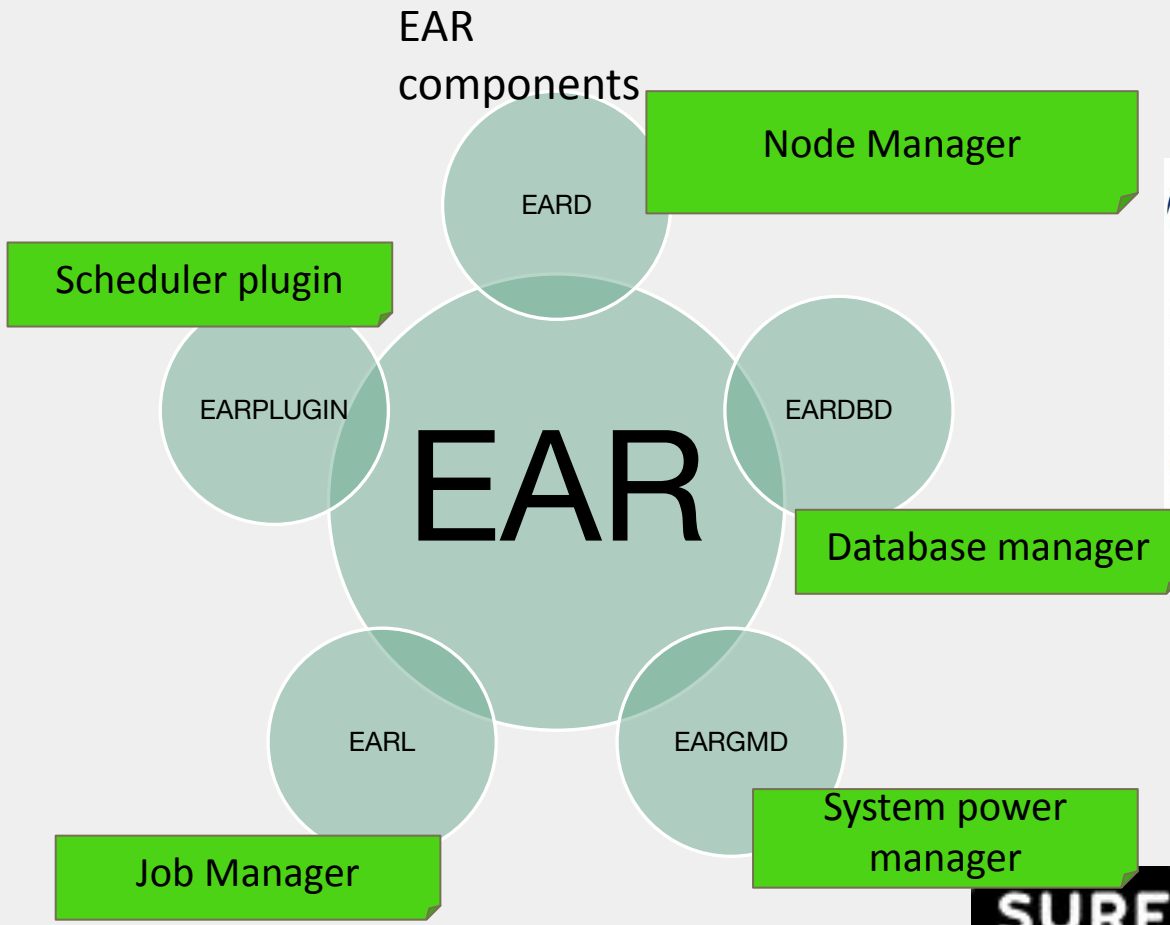
Optimize my app

I want to improve my app

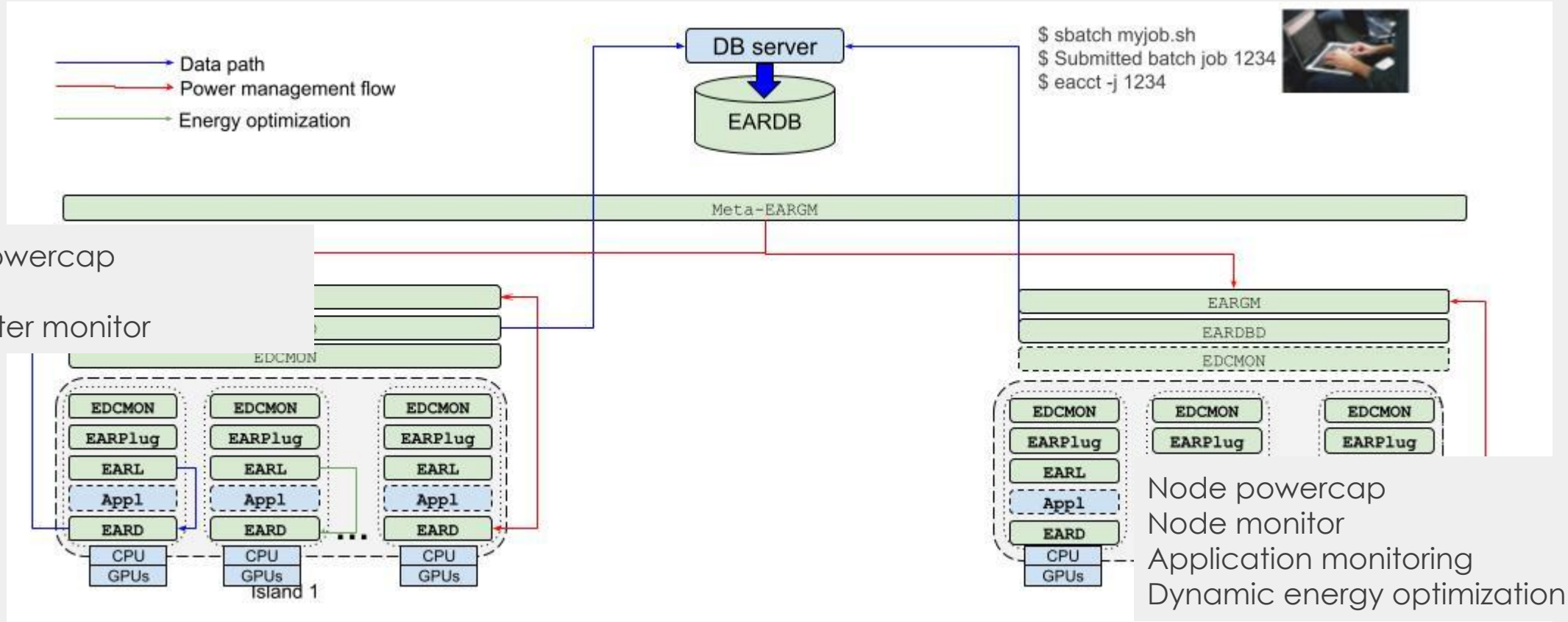


EAR Goals and Components

- To be
  - Easy to use
  - Transparent for users
  - Powerful for developers
- To optimize energy at runtime
- To be flexible and configurable
- Power management



EAR architecture



Cluster powercap
DB report
Data center monitor

Node powercap
Node monitor
Application monitoring
Dynamic energy optimization

- Software group of nodes
- Distributed features: Monitoring, optimization, power management, reporting
- Vendor independent

EAR
Energy Aware Runtime

Snellius

Dutch National Supercomputer

Barcelona Supercomputing Center
Centro Nacional de Supercomputación

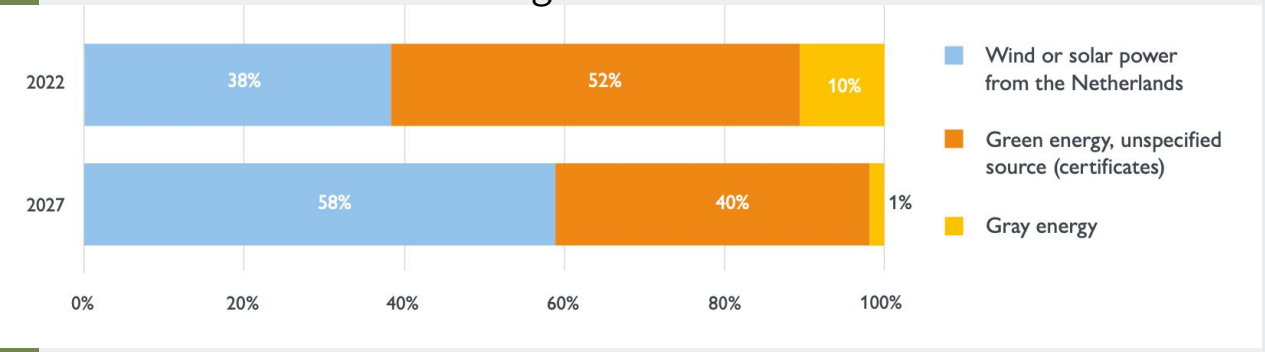
SURF

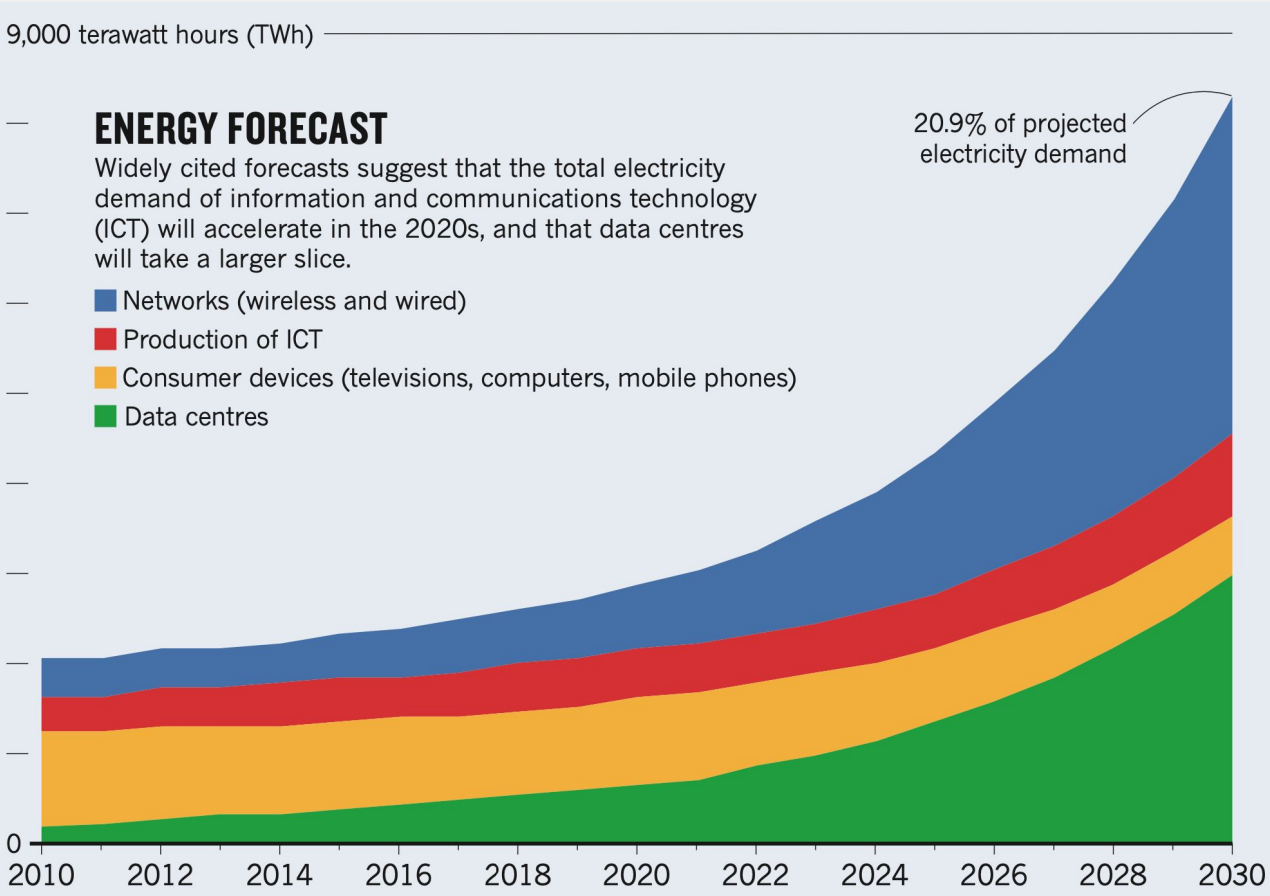
- Jones, Nicola. "How to stop data centres from gobbling up the world's electricity." *Nature* 561.7722 (2018): 163-166.
- Andrae, Anders SG, and Tomas Edler. "On global electricity usage of communication technology: trends to 2030." *Challenges* 6.1 (2015): 117-157.
- Bakkeren, Hanno. "Datacenters verbruiken drie keer zoveel stroom als de NS". NRC, 14 May 2019
- State of the Dutch Data Centers, The Dutch Data Center Report, 2022, https://www.dutchdatacenters.nl/

EAR Energy Aware Runtime

ICT/Data center energy forecast

- Data centre energy usage:
  - ~ 200 TWh for data centres in 2018
  - ~ 3000 TWh in 2030
- Dutch Data Center Usage (2019)
  - 1300 MW installed capacity
  - 0.3 % electricity usage of the Netherlands (CBS)
  - 3x as much power than the NS
- Dutch Data center energy consumption (2022)
  - 90% of all energy consumed by colocation data centers is green.

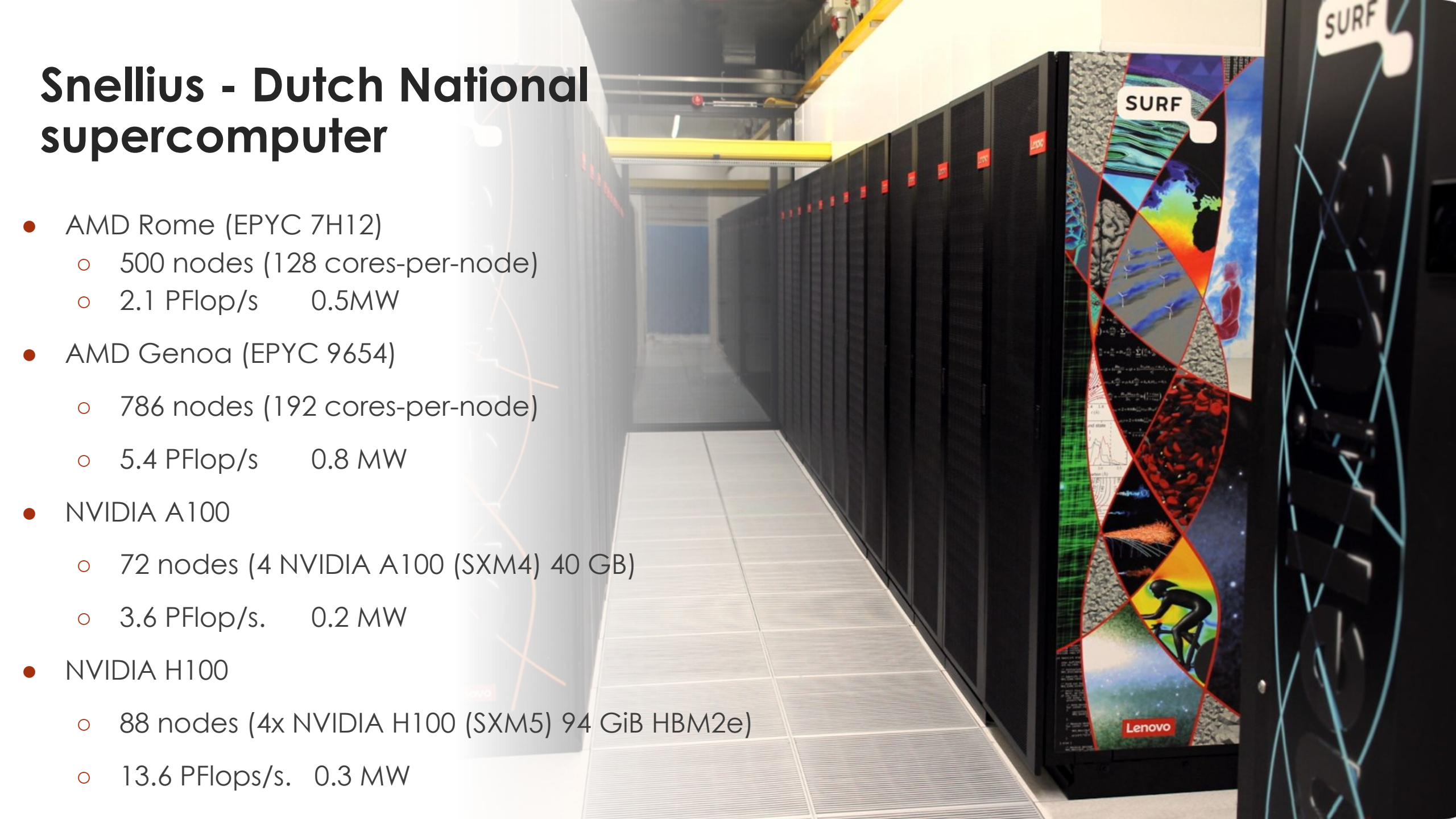




The chart above is an 'expected case' projection from Anders Andrae, a specialist in sustainable ICT. In his 'best case' scenario, ICT grows to only 8% of total electricity demand by 2030, rather than to 21%.

Snellius - Dutch National supercomputer

- AMD Rome (EPYC 7H12)
  - 500 nodes (128 cores-per-node)
  - 2.1 PFlop/s 0.5MW
- AMD Genoa (EPYC 9654)
  - 786 nodes (192 cores-per-node)
  - 5.4 PFlop/s 0.8 MW
- NVIDIA A100
  - 72 nodes (4 NVIDIA A100 (SXM4) 40 GB)
  - 3.6 PFlop/s. 0.2 MW
- NVIDIA H100
  - 88 nodes (4x NVIDIA H100 (SXM5) 94 GiB HBM2e)
  - 13.6 PFlops/s. 0.3 MW



Top 500 (Nov-2024)

1.El Capitan-US (30 MW) 1,742.0 PFlops

2.Frontier-US (24 MW) 1,353.0 PFlop/s

3. Aurora-US (38 MW) 1,012.0 Pflop/s

4. Eagle-US, Microsoft (??) 561 Pflop/s

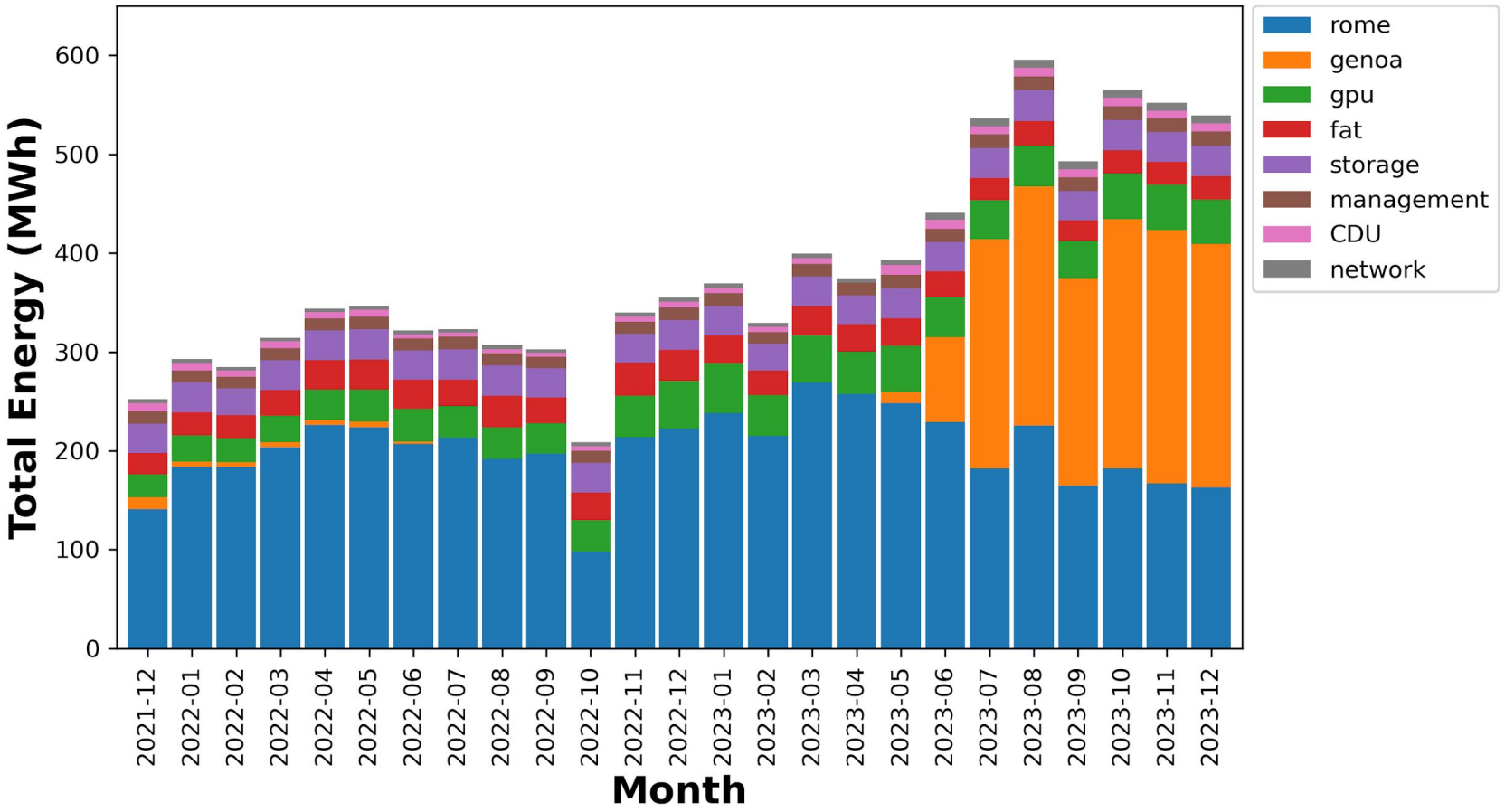
5. HP6 (8 MW) 477.01 PFlop/s

Snellius

(#197) CPU: LINPACK Rmax (0.8 MW) 5.4 PFlop/s

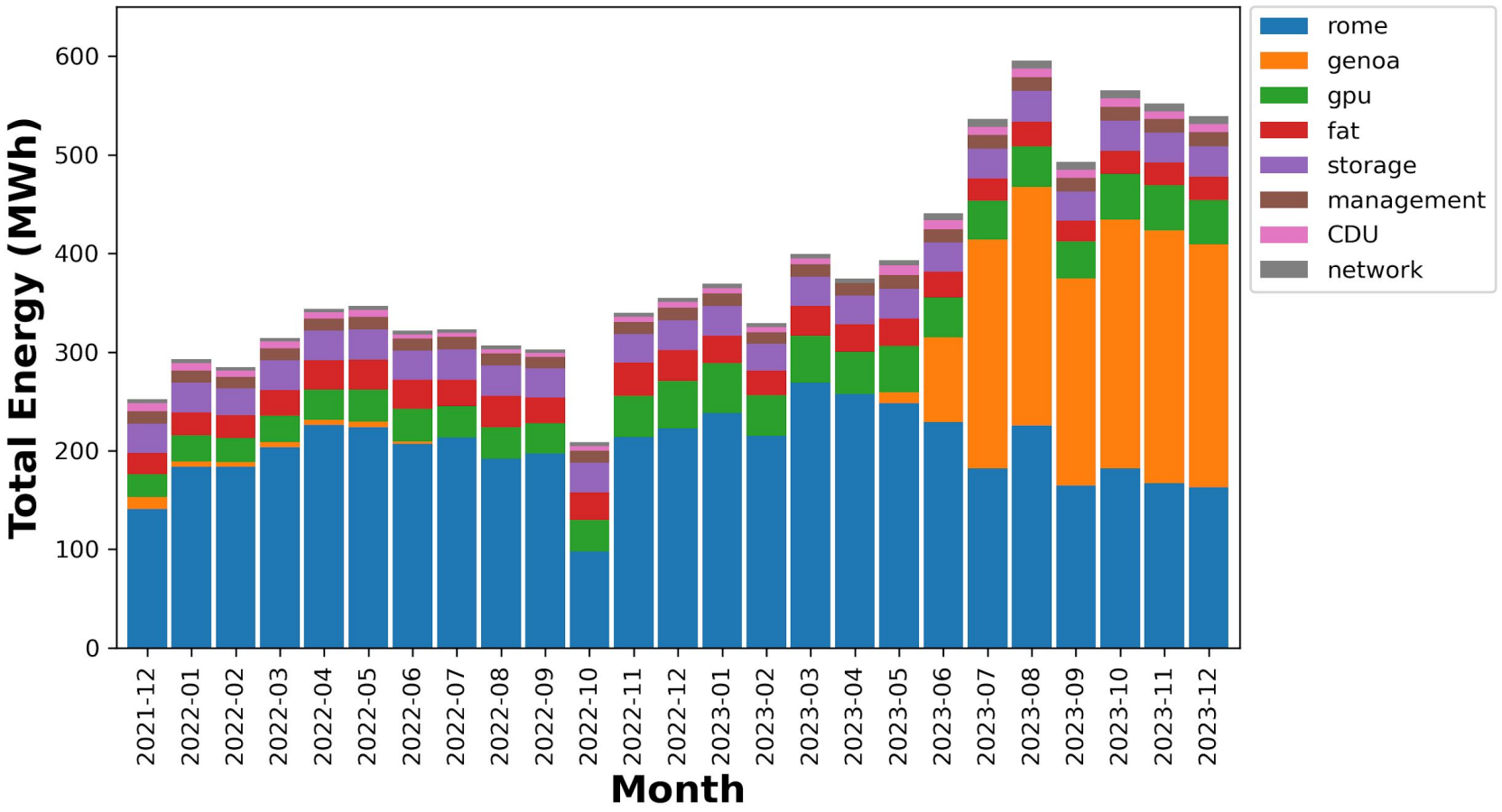
(#94) GPU: LINPACK Rmax (0.3 MW) 13.64 PFlop/s

Snellius - Total Energy Usage



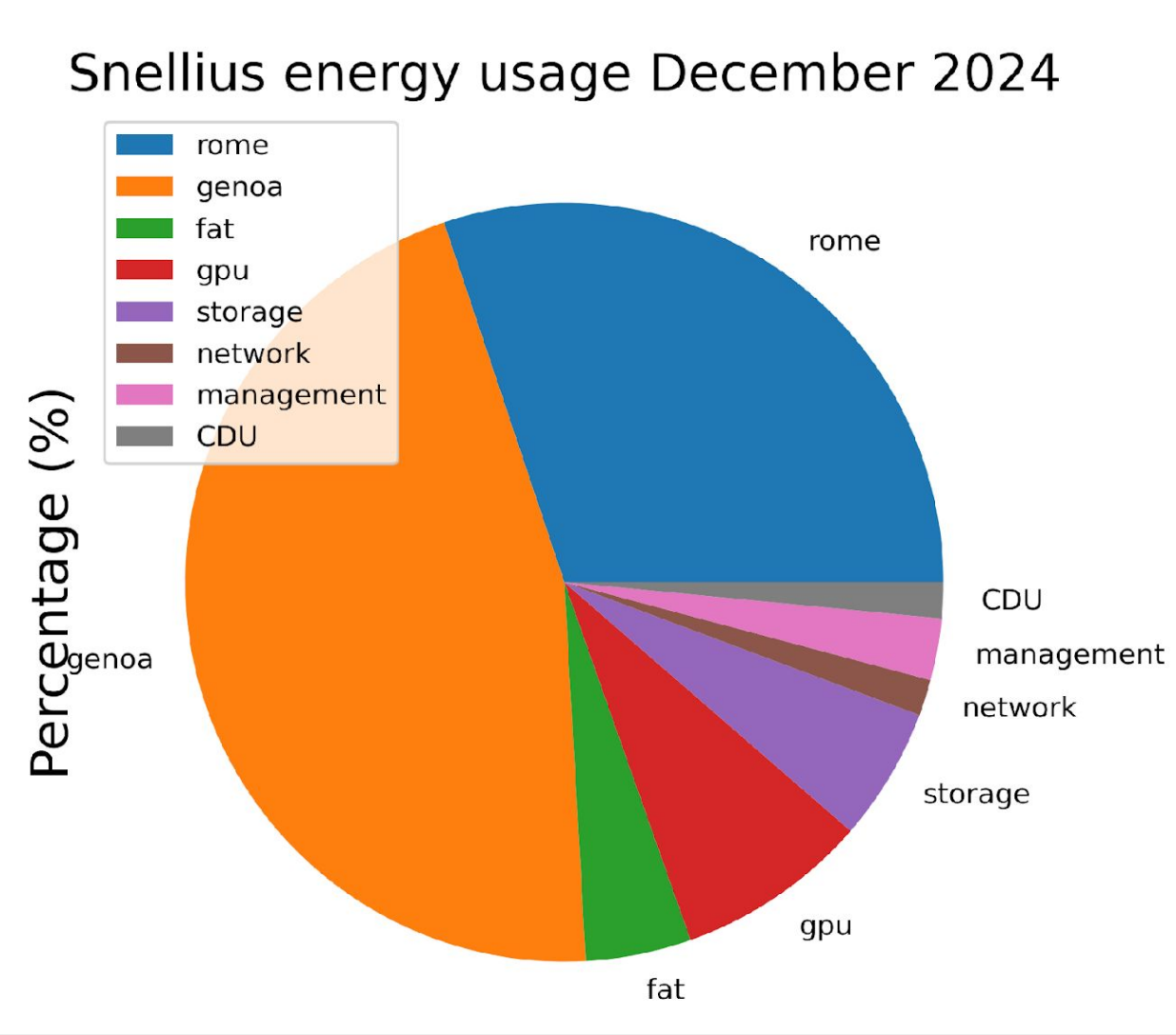
- Snellius in December 2023 used ~600MWh
- ~2500 house holds worth

Snellius - Total Energy Usage



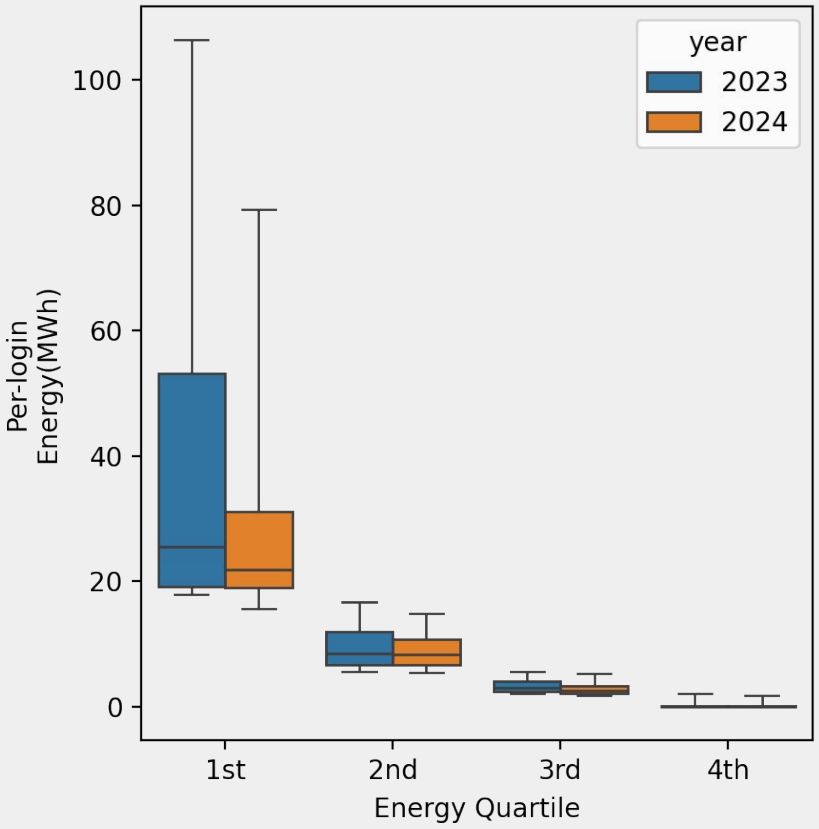
- 88% Compute
  - 80% CPU
    - 34% Rome (3% fat)
    - 45% Genoa
  - 8% Gpu
- 12% Other
  - 5.0% Storage
  - 2.6% Management
  - 1.5% Cooling
  - 1.5% Network

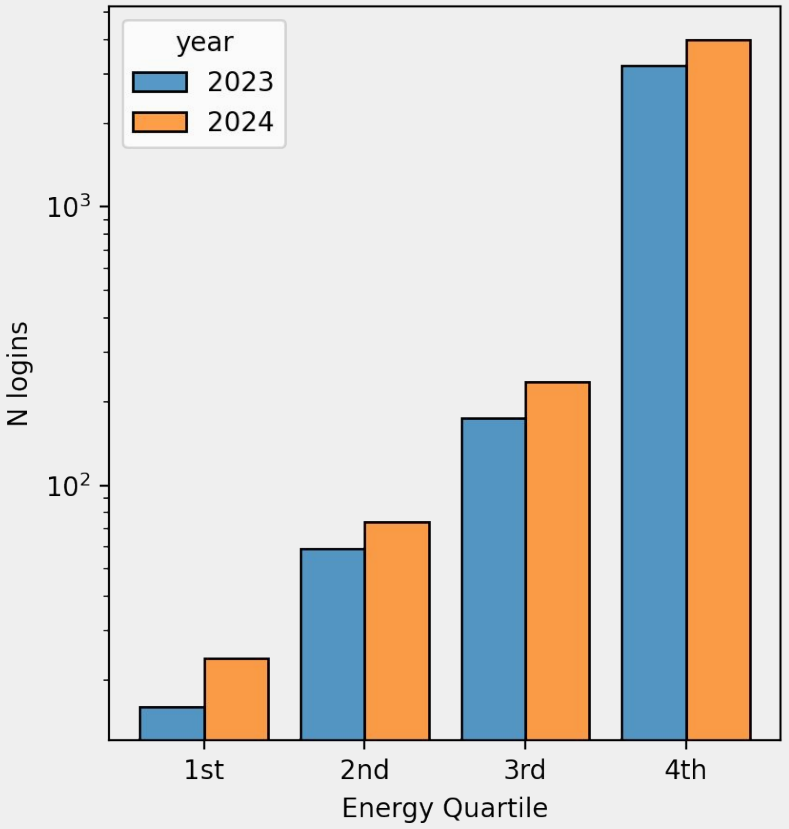
Snellius - Energy usage per partition



- 88% Compute
  - 80% CPU
    - 34% Rome (3% fat)
    - 45% Genoa
  - 8% Gpu
- 12% Other
  - 5.0% Storage
  - 2.6% Management
  - 1.5% Cooling
  - 1.5% Network

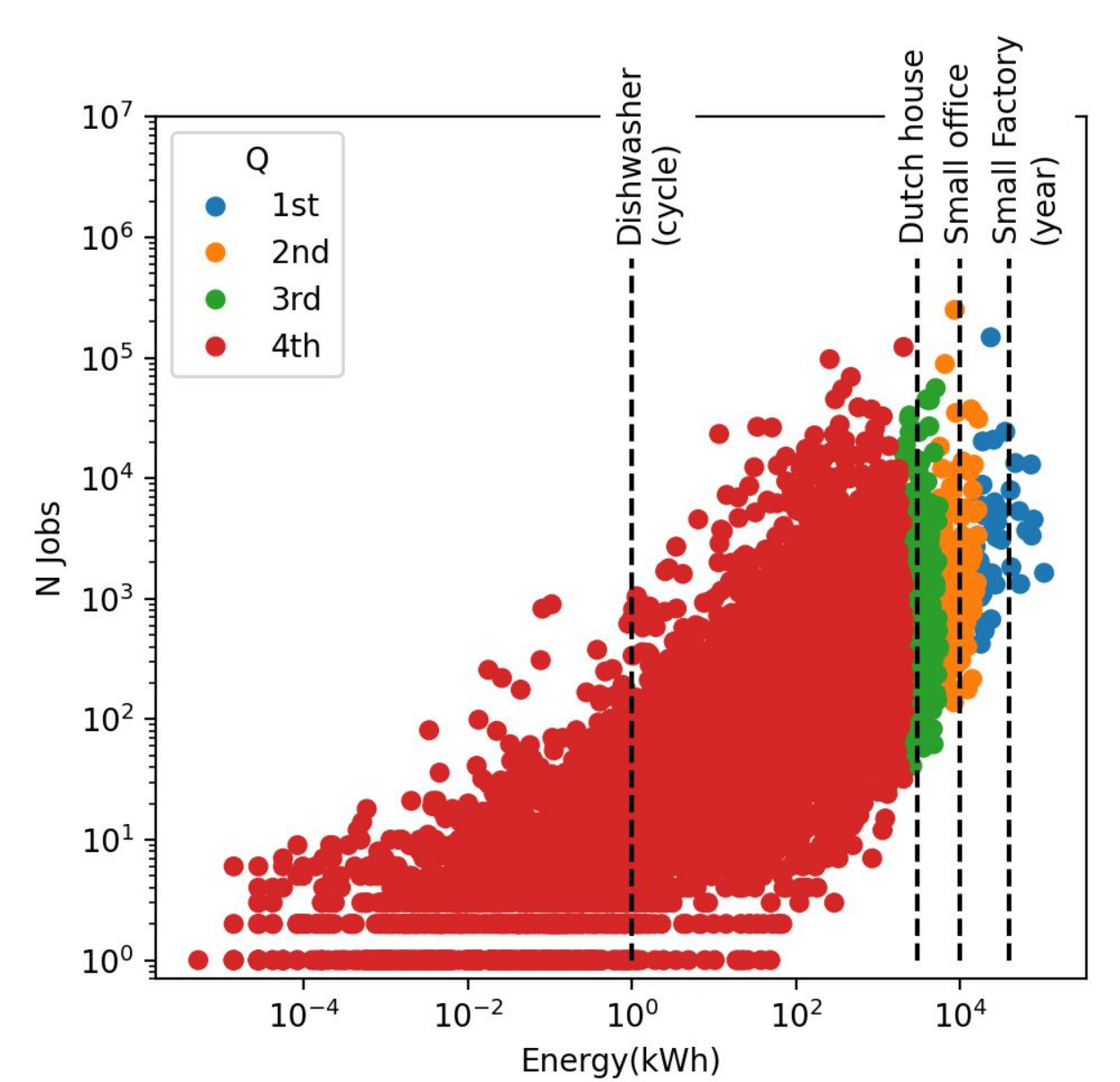
Snellius - end user energy consumption





- 2.3 GWH by users per year
- Q1 group: 15 logins (0.4%)
- Q2 group: 74 logins (2.1%)
- Q3 group: 249 logins (7.1%)
- Q4 group: 3145 logins (90.3 %)

Snellius - end user submission trends



- 2.3 GWH by users per year
- Q1 group: 15 logins (0.4%)
- Q2 group: 74 logins (2.1%)
- Q3 group: 249 logins (7.1%)
- Q4 group: 3145 logins (90.3 %)

EAR components

EARD: Computational Node Manager

- Linux service running in all the compute nodes (1 x compute node)
- With **root privileges**
- EARD offers
  - **Node monitoring**
  - **Basic Job accounting**
  - **Node powercap**
  - API for local metrics readings and management operations (used by EAR library)
  - API for external power and management
- Applies energy settings described in ear.conf
  - Default policy, frequency
  - Controls EAR privileged users/groups/accounts

EARDBD: Data Base manager

- Linux service running in service nodes
- **Data Base manager**. Connects with DB server
- 1..N EARDBD can be run in the system
- EARDBD aggregates power information
  - Each EARDBD aggregates metrics for all the EARDs contacting with it for a specified period (ear.conf)
  - Each EARDBD reports data to DB in batch messages reducing the number of connections and the cost of queries

EARL: EAR optimization library

- User-level Runtime library
- 100% transparent in most of the use cases (for example MPI applications)
- Application energy and performance monitoring
- Application dynamic energy optimization
- Extensible reporting mechanism
    - Multiple report plugins can be used: DB, CSV, etc

EARGMD: Global/System Power Manager

- System Power manager
- Energy&Power monitoring for the whole cluster
- Cluster powercap

Scheduler plugin

- SLURM SPANK plugin, PBS and OAR supported
  - Next release with K8 support
- Intercepts job/step creation and
  - connects with EARD to report scheduling events : new job / end job for example
  - Configures the environment for the automatic execution of EAR and environment variables

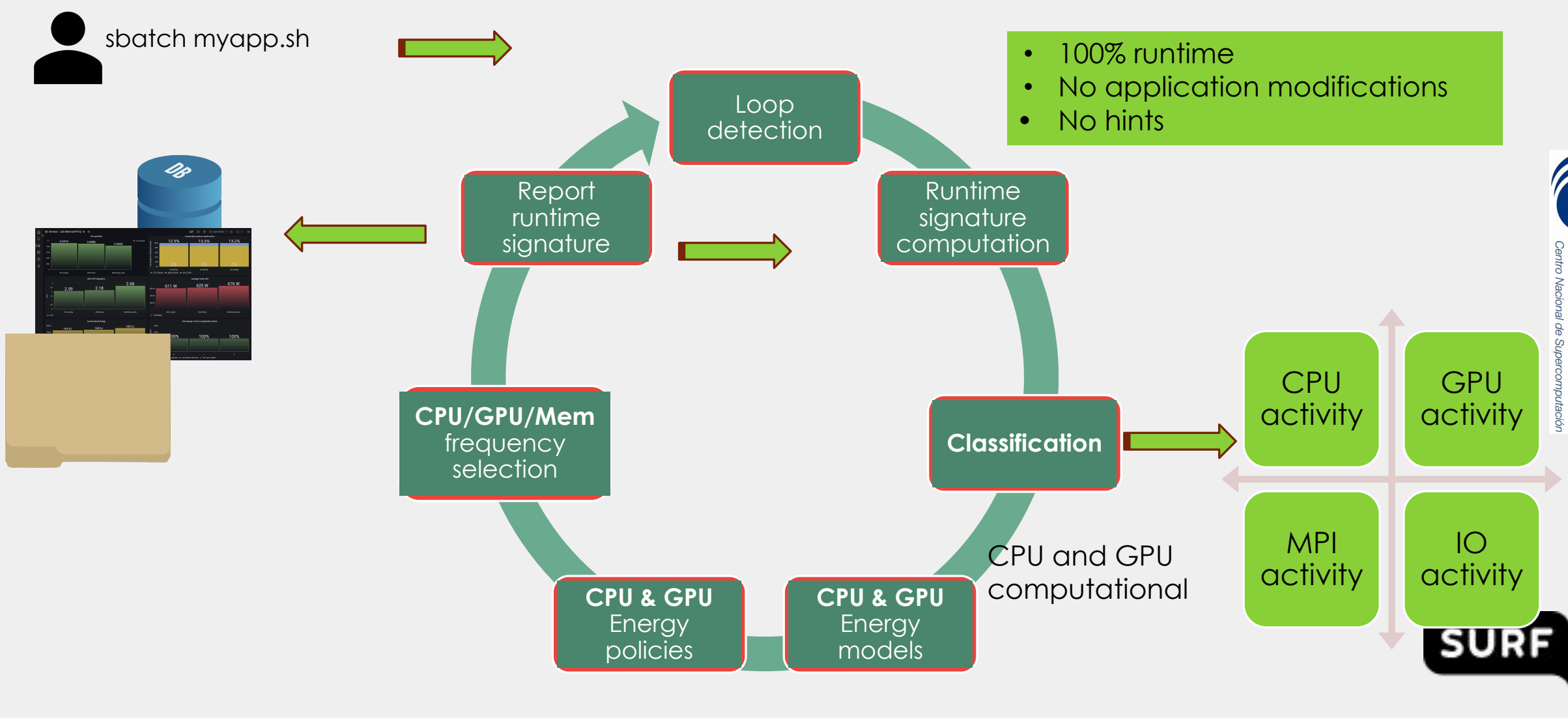
EAR (runtime) Library

Performance metrics and energy optimization

EAR Library

- User-level Runtime library
- Transparent to users through scheduler plugins
  - Few exceptions need some environment variables: Ex. Singularity
- Application energy and performance monitoring
- Application dynamic energy optimization
- Extensible design based on plugins
  - Multiple report plugins can be used: DB, CSV, etc
  - New policies
  - New energy models
  - ETC

The EAR Library optimization loop



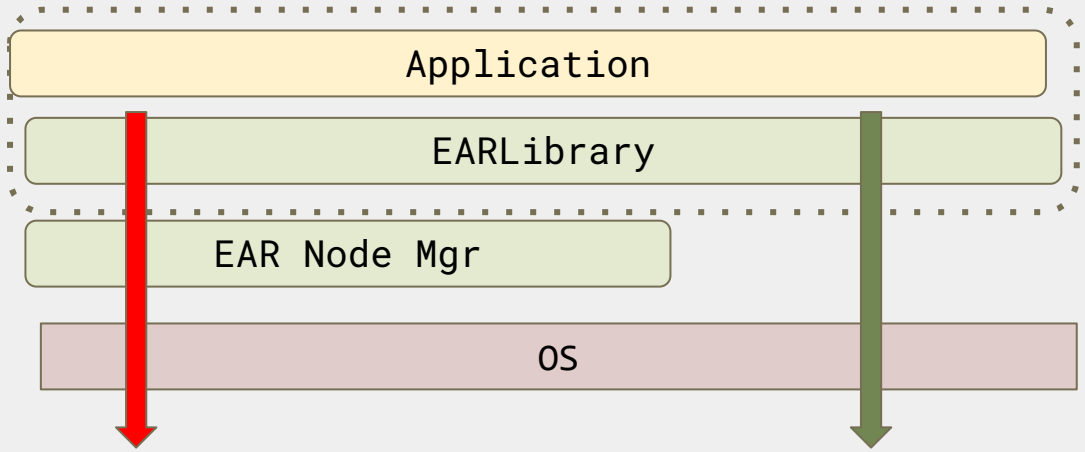
- 100% runtime
- No application modifications
- No hints

Is it really easy to use? YES

- In systems where EAR is fully installed, EAR includes submissions plugins to be integrated in the scheduler
  - **srun –ear=on my_application**
  - SLURM and PBS pro tested in production
  - K8 in progress
- What if my data center doesn't want to install EAR??
  - You can still use it, EAR can be installed in user mode only (we call it EAR-lite)
  - erun command with same SLURM flags: **erun –program="my_application arg1..argn"**
  - Only the EAR runtime
  - Very powerful for Application monitoring
  - We have used in : Intel, AMD, Grace, Fujitsu, … NVIDIA, Intel PVC, AMD GPU…. Lenovo, HPE- Cray etc
- Does EAR-lite has some limitations?
  - Yes, some metrics are not available since require root privilege
  - Optimization is not supported since require root privilege

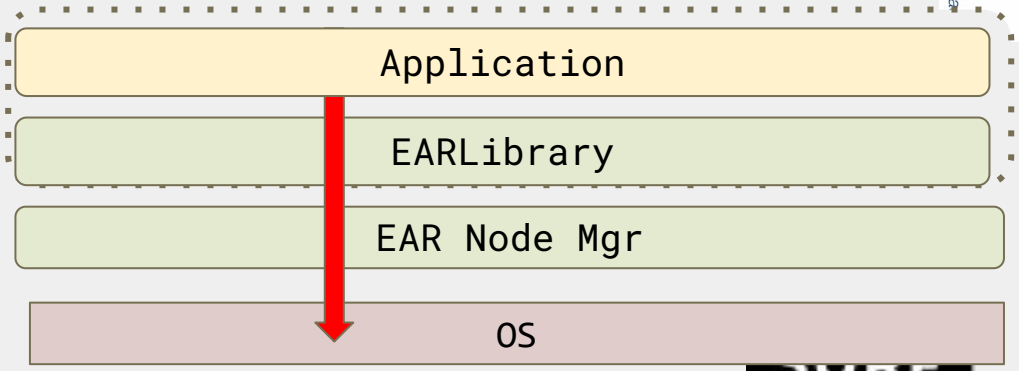
Is it portable?

- EAR is very portable and flexible since all the APIs for monitoring and management are implemented in **two layers**
  - Generic
  - Device/API specific
- There is always a **dummy API** in case one API is not supported for a new architecture



IPMI RAPL MSR HSMP REDFISH etc

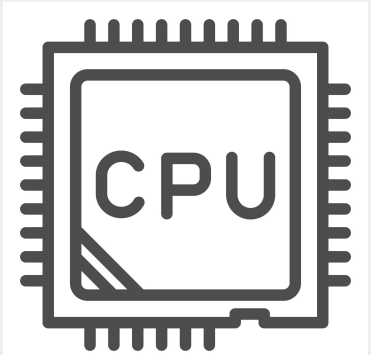
Perf nvidia levelzero rsmi hwmon etc



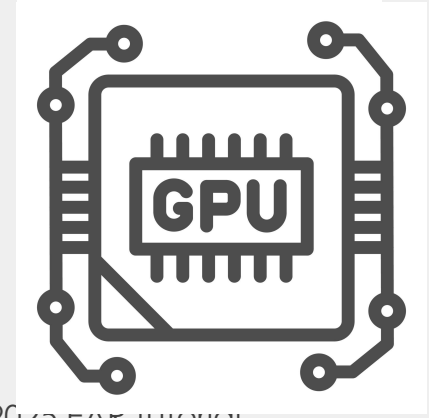
- ➔ OS Cpufreq : userspace, intel_pstate
- ➔ AMD HSMP with virtual list of p-states
- ➔ GPU
  - Libnvdidia-ml Levelzero, smi

And what about the data: the signature

- EAR IDs are
  - Job, step, nodename and "application", where Application is EAR specific and the others are scheduler specific
- The scope is : Application and runtime



CPU activity : CPI, Memory BW, GFlops, IO, MPI statistics
Power consumption: DC node power, DRAM and CPU power



GPU activity : GPU and memory utilization, GPU flops (NVIDIA)
Power consumption: DC node power, GPU power

See our wiki for the full list :
https://gitlab.bsc.es/ear_team/ear/-/wikis/EAR-Database

Running jobs with EAR

Job submission use cases

- Default use cases + schedule support → automatic (--ear=on)
- Not default: Use EAR_LOADER_APPLICATION=my_application

	Use case	Bootstrap	Automatic
MPI	Intel/OpenMPI [+ others]	srun	yes
	Intel [+ others]	mpirun	yes
	OpenMPI [+ others]	mpirun	no : use erun
OpenMP		srun	yes (or use erun)
CUDA		srun	yes (or use erun)
python		srun	yes (or use erun)
Singularity		srun	with module or env var support
julia		srun	yes

https://gitlab.bsc.es/ear_team/ear/-/wikis/User-guide#use-cases

Job submission + EAR library options

- Many of the job job scripts will work without modification.
- EAR flags in "help"

```
[julitac@int1 ~]$ sbatch --help|grep ear
      --ear=on|off                Enables/disables Energy Aware Runtime Library (default OFF)
      --ear-policy=policy          Selects an energy policy for EAR (monitoring, min_energy)
      --ear-cpufreq=frequency Specifies the CPU frequency to be used by EAR , to be used with  monitoring
     --ear-user-db=file           Specifies the file to save the job applications metrics (csv format)
      --ear-verbose=value         Specifies the level of verbosity (0 default)
```

Memory and GPU frequency also supported with environment variables. See wiki

Special cases (I)

- **OpenMPI**
  - srun recommended (100% automatic)
  - mpirun requires erun support (ear command)
- **Singularity**: EAR can be used but EAR paths and env vars must be exported: APPTAINER_ENV_XXX, APPTAINER_BIND
  - export APPTAINER_BIND="$EAR_INSTALL_PATH:$EAR_INSTALL_PATH:ro,$EAR_TMP:$EAR_TMP:rw"
  - export APPTAINERENV_EAR_INSTALL_PATH=$EAR_INSTALL_PATH
  - export APPTAINERENV_EAR_TMP=$EAR_TMP
  - export APPTAINERENV_EAR_ETC=$EAR_ETC

Monitoring: EAR metrics

https://gitlab.bsc.es/ear_team/ear/-/wikis/EAR-commands#ear-job-accounting-eacct

Monitoring

- Jobs executed without EAR library (ear = off) report **basic job/step accounting**
  - Job/step/node identification
  - Job/step/node execution time, energy consumption, average CPU freq.

- Jobs executed with EAR library (ear =on) report **advanced application accounting**
  - Job/step/**application**/node identification
  - Job/step/**application**/node performance metrics (measured by EAR library)
  - Job/step/**application**/node power metrics (measured by EAR library)

- Data is reported in EAR DB

EAR Signature

- Node metrics: DC node power
- CPU metrics
  - Performance: CPI, memory bandwidth, IO, %MPI , CPU utilization, FLOPs, Cache , Avg CPU frequency, AVG memory frequency, temperature
    - Network → not yet (ongoing)
  - Power: CPU and DRAM
- GPU metrics
  - GPU util, MEM util, Power, temperature, Average frequency, GPU flops
  - GPU power consumption
  - On demand: DCGMI events

- https://gitlab.bsc.es/ear_team/ear/-/wikis/EAR-Database + EARL specific metrics (GPU "extra" metrics"

How to get application data

1. **With EAR job accounting command eacct**: Command line with pre-defined queries. Multiple filters supported
    a. average
    b. per node
    c. runtime
    d. pre-selected column in stdout or full data in CSV file
2. **Directly from EAR library**
    a. csv with timestamp included: **--ear-user-db=filename** (prefix for the file)
    b. Additional report plugins can be used with env var.
        i. **EAR_REPORT_ADD**=plugin1.so:plug2.so

Example: https://gitlab.bsc.es/ear_team/ear/-/blob/master/src/report/log.c

Basic MPI example

```
#!/bin/bash

#SBATCH -p rome
#SBATCH -t 00:15:00
#SBATCH --nodes=1
#SBATCH --exclusive

#SBATCH --output=NPB.%j.out
#SBATCH --error=NPB.%j.err
#SBATCH --job-name=NPB

module load 2023
module load foss/2023a

PROJECT_DIR=/projects/0/energy-course

srun –ear=on --ntasks=128 $PROJECT_DIR/NPB3.4-MZ-MPI/sp-mz.C.x
```

Job submission examples

```
#!/bin/bash
#SBATCH –ntasks=YYY
#### EAR=ON will load all the steps with EAR library
#SBATCH --ear=on

mkdir -p logs
# CASE 1: Default: EAR library on because of headers
srun application
# Runtime metrics reported ON
export EARL_REPORT_LOOPS=1
# CASE 2:  mpirun + ear-user-db → CSV file
export I_MPI_HYDRA_BOOTSTRAP_EXEC_EXTRA_ARGS="--ear-user-db=logs/app"
mpirun application
# CASE 3: Using srun + ear-user-db → CSV file
srun --ear-user-db=logs/bt.srun application
# CASE 4: Using erun
module load ear
mpirun -n XXXX  erun --ear=on --program="application arg1 arg2…argn”
# CASE 5: EAR library off for this steps
srun --ear=off application
```

eacct: Energy accounting

- SLURM jobid/stepid
- Users can only access its own data
- GPU support is per-cluster, Jobs executed in AMD partition will also show GPU metrics with null values.
- By default, average per job.step metrics: All nodes included. Most metrics are averaged, energy is accumulated.
- Main flags:
  - -j → job id selection
  - -l → per node
  - -r → runtime metrics (default is off in snellius. Use **export EARL_REPORT_LOOPS=1**
  - -c filename → save in CSV format in file

```
[julitac@int3 example]$ eacct -j 1483484
JOB-STEP USER      APPLICATION      POLICY NODES AVG/DEF/IMC(GHz) TIME(s)   POWER(W) GBS    CPI     ENERGY(J)   GFLOPS/W IO(MBs) MPI% G-POW(T/U)  G-FREQ
G-UTIL(G/MEM)
1483484-sb julitac   128nodes_16cores NP   16   2.35/2.60/---    972.00    375.71   ---    ---     5843040     ---      ---     ---   ---      ---     ---
1483484-1  julitac   128nodes_16cores MT   16   2.35/2.40/1.47   492.26    373.53   28.44  0.46    2942015     0.0084   250.8   71.7  0.00/---    ---     ---
1483484-0  julitac   128nodes_16cores ME   16   2.55/2.60/1.47   460.68    381.00   30.37  0.47    2808271     0.0088   268.2   71.7  0.00/---    ---     ---
```

Tensorflow: GPU application

```
[julitac@int5 ~]$ eacct -j 5687690
   JOB-STEP USER      APPLICATION      POLICY NODES AVG/DEF/IMC(GHz) TIME(s)    POWER(W) GBS    CPI   ENERGY(J)    GFLOPS/W IO(MBs) MPI%  G-POW
(T/U)    G-FREQ  G-UTIL(G/MEM)
5687690-sb   julitac    run_tensor.sh    NP       1    2.43/2.40/---   2612.00   448.98  ---    ---   1172725     ---      ---    ---   ---          ---    ---

5687690-8    julitac    DenseNet121_disa MO       1    2.38/2.40/2.19   358.55    897.17  2.03   0.66  321685      0.0000   0.1    0.0   257.92 /257.92  1.410  92%/44%
5687690-7    julitac    DenseNet121_mixe MO       1    2.38/2.40/2.19   189.30    876.59  0.82   0.67  165936      0.0000   0.2    0.0   238.88 /238.88  1.410  80%/52%
5687690-6    julitac    DenseNet121      MO       1    2.38/2.40/2.19   249.38    890.48  0.59   0.66  222070      0.0000   0.1    0.0   251.56 /251.56  1.410  88%/73%

5687690-5    julitac    VGG19_disable-tf MO       1    2.38/2.40/2.19   646.86    877.18  2.33   0.52  567419      0.0000   0.1    0.0   238.58 /238.58  1.410  98%/26%
5687690-4    julitac    VGG19_mixed      MO       1    2.38/2.40/2.19   248.40    897.64  0.52   0.67  222972      0.0000   0.1    0.0   257.04 /257.04  1.410  96%/51%
5687690-3    julitac    VGG19            MO       1    2.38/2.40/2.19   261.37    907.44  3.21   0.61  237176      0.0000   0.1    0.0   267.94 /267.94  1.410  96%/59%

5687690-2    julitac    ResNet50_disable MO       1    2.38/2.40/2.19   302.42    920.38  4.22   0.65  278338      0.0000   0.1    0.0   279.54 /279.54  1.410  94%/43%
5687690-1    julitac    ResNet50_mixed   MO       1    2.38/2.40/2.19   149.26    873.35  0.80   0.66  130356      0.0000   0.2    0.0   233.60 /233.60  1.403  84%/50%
5687690-0    julitac    ResNet50         MO       1    2.38/2.40/2.19   196.33    904.09  0.62   0.65  177504      0.0000   0.2    0.0   261.45 /261.45  1.372  87%/70%
```

EACCT metrics: what else can be observed?

- Are nodes homogeneous ? Same signatures in all the nodes (-l)
- Are there phases, is it constant? Are runtime signatures the same? (-r)
- How much time my application is in MPI calls??
- GPU utilization
- Impact on power and performance of changing job configuration
  - Example: Problem with GPU utilization in GROMACS-GPU

```
[julitac@int3 ~]$ eacct -j 1387089
JOB-STEP USER      APPLICATION      POLICY NODES AVG/DEF/IMC(GHz) TIME(s)   POWER(W) GBS    CPI  ENERGY(J)   GFLOPS/W IO(MBs) MPI%  G-POW
(T/U)   G-FREQ  G-UTIL(G/MEM)
1387089-sb  julitac   rfm_GROMACS_GPU_ NP     1    2.35/2.40/---   6634.00   879.52  ---   ---  5834703     ---      ---    ---  ---          ---    ---
1387089-2   julitac   rfm_GROMACS_GPU_ MT     1    2.35/2.20/2.02  2886.16   803.91  24.39 0.37 2320201     0.0091   0.1    71.0 333.42/333.42 1.409  10%/0%
1387089-1   julitac   rfm_GROMACS_GPU_ ME     1    2.35/2.40/2.10  2880.70   847.76  37.46 0.38 2442131     0.0129   0.1    68.1 359.00/359.00 1.409  14%/0%
1387089-0   julitac   rfm_GROMACS_GPU_ MO     1    2.38/2.40/2.19  833.64    1260.21 152.87 0.56 1050561     0.0425   0.4    26.4 650.88/650.88 1.409  73%/0%
```

Data visualization: ear-job-visualizer (SNELLIUS)

Option 1: Load the software module on Snellius

- module load 2024
- module load ear-job-analytics/5.0-gfbf-2024a

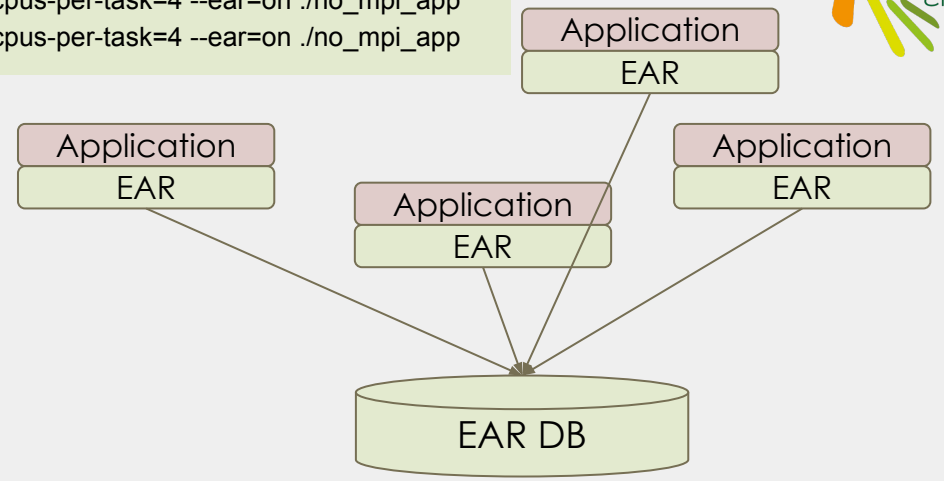
Option 2: git clone …. pip install

- https://github.com/eas4dc/ear-job-visualization

Monitoring

export EAR_NTASK_WORK_SHARING=1 → Group tasks by jobid.stepid

```
srun --cpu-bind=verbose -J master4   -u --ntasks=4  --cpus-per-task=4 --ear=on ./no_mpi_app
srun --cpu-bind=verbose -J master20 -u --ntasks=20  --cpus-per-task=4 --ear=on ./no_mpi_app
srun --cpu-bind=verbose -J master30 -u --ntasks=30  --cpus-per-task=4 --ear=on ./no_mpi_app
```



Node sharing model

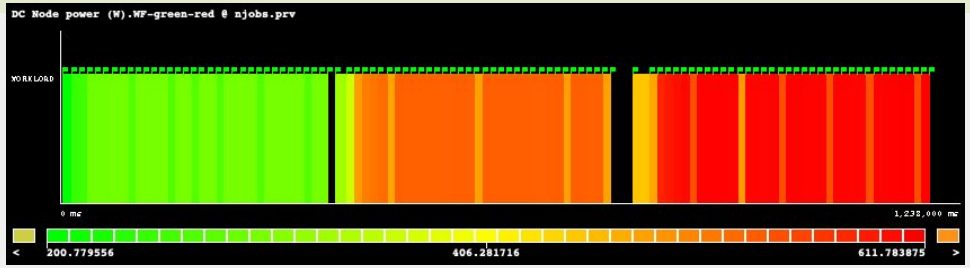
sbatch **–ear=on** my_job.sh
Submitted batch job 11808826

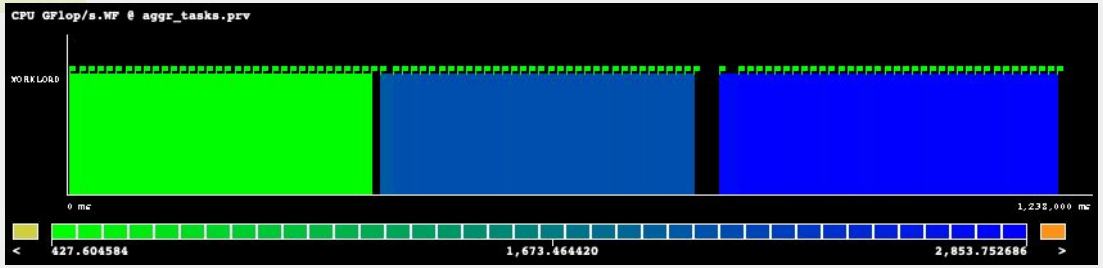
eacct -j 11808826

```
[julitac@int5 Multiprocess]$ eacct -j 11808826
 CPU busy waiting  memory bound  cpu bound  IO bound
    JOB-STEP AID    USER       APPLICATION      POLICY NODES AVG/DEF/IMC(GHz) TIME(s)     POWER(W) GBS     CPI   ENERGY(J)    GFLOPS/W IO(MBs) MPI%  G-POW (T/U)     G-FREQ  G-UTIL(G/MEM)
11808826-sb   0      julitac    N_ear.sh         MO     1     2.42/2.60/---    1238.00     465.26   ---     ---   575995       ---      ---     ---   ---              ---     ---
11808826-2    3496199 julitac    master30         MO     1     2.39/2.60/1.47   445.64      588.33   128.63  0.37  262182       4.8951   0.0     0.0   0.00/---         ---     ---
11808826-1    3495416 julitac    master20         MO     1     2.57/2.60/1.47   403.43      515.24   94.94   0.37  207865       4.0797   0.0     0.0   0.00/---         ---     ---
11808826-0    3492846 julitac    master4          MO     1     2.57/2.60/1.47   372.22      284.97   20.72   0.34  106071       1.5500   0.0     0.0   0.00/---         ---     ---
```

sbatch **–ear-policy=min_energy** my_job.sh
Submitted batch job 11808827

ear-job-visualizer -j 11808826 --format ear2prv -o traces/njobs





Monitoring: per application

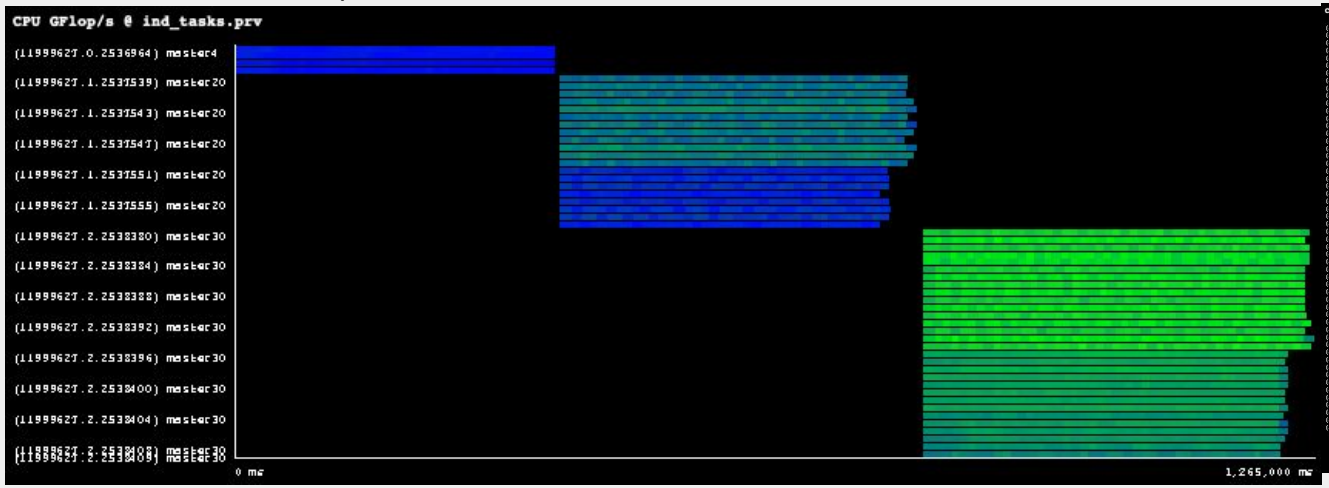
sbatch **–ear=on** my_job.sh
Submitted batch job 11999627

export EAR_NTASK_WORK_SHARING=0 → Each task is considered one application

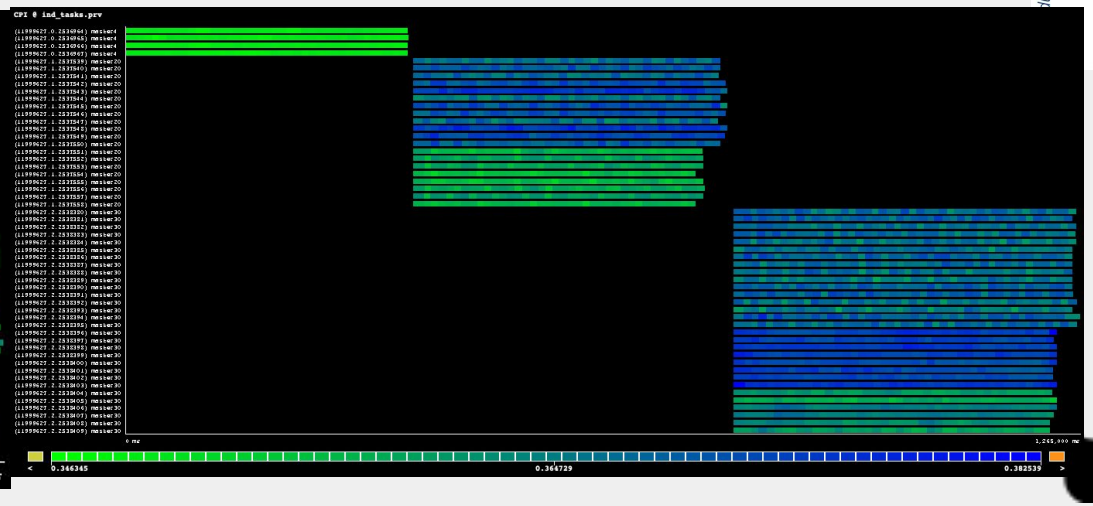
```
srun --cpu-bind=verbose -J master4   -u --ntasks=4   --cpus-per-task=4 --ear=on ./my_app
srun --cpu-bind=verbose -J master20 -u --ntasks=20  --cpus-per-task=4 --ear=on ./my_app
srun --cpu-bind=verbose -J master30 -u --ntasks=30  --cpus-per-task=4 --ear=on ./my_app
```

```
ear-job-visualizer -j 11999627 --format ear2prv -o traces/njobs
```

Per-task CPU Gflops



Per-task CPU CPI



EAR & PyCOMPSs: Coarse grain integration (submission + tracing)

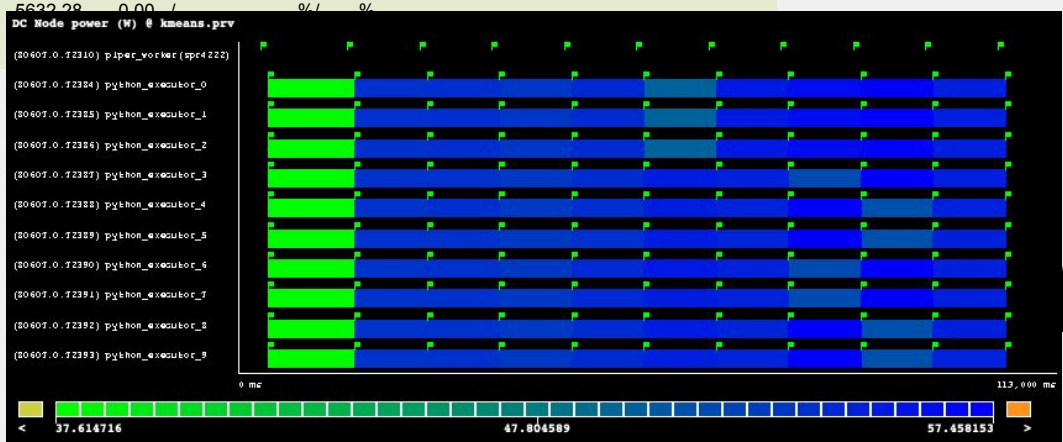
launch.sh

```
enqueue_compss --num_nodes=1 \
--qos=miscspr \
--tracing=true \
--exec_time=20 \
--worker_working_dir=$(pwd) \
--keep_workingdir \
--pythonpath=$(pwd) \
--ear="true" \
$(pwd)/kmeans.py -n 102400000 -f 32 -d 60 -c 8 -i 100
```

Automatic submission with EAR

```
[xjcorbalan@login4601 COMPSs]$ eacct -j 80607
    JOB-STEP AID      USER      APPLICATION     POWER(W)  NODES    AVG/DEF/IMC(GHz) TIME(s)  GBS    CPI  GFLOPS/W  ENERGY(J)   G-POW (T/U)    G-FREQ  G-UTIL(G/MEM)
G-GFLOPS
  80607-sb  0        xjcorbalan COMPSs         579.56    1       ---/---/---      113      579.56    ---   ---  ---       65490.02    --    /--     ---       ---%/---%    ---
  80607-0   72310    xjcorbalan piper_worker(spr 0.00      1       3.00/2.00/2.41   109      0.00    1.76  0       0.00        0.00    /--     ---       ---%/---    % ---
  80607-0   72385    xjcorbalan python_executor_ 52.00     1       2.99/2.00/2.41   108      9.55    1.00  0.02    5631.24     0.00    /--     ---       ---%/---    % ---
  80607-0   72393    xjcorbalan python_executor_ 52.00     1       3.00/2.00/2.41   108      7.17    1.00  0.02    5631.87     0.00    /--     ---       ---%/---    % ---
  80607-0   72392    xjcorbalan python_executor_ 52.00     1       2.99/2.00/2.41   108      9.30    1.00  0.02    5630.51     0.00    /--     ---       ---%/---    % ---
  80607-0   72391    xjcorbalan python_executor_ 52.00     1       3.00/2.00/2.41   108      7.56    1.00  0.02    5632.75     0.00    /--     ---       ---%/---    % ---
  80607-0   72390    xjcorbalan python_executor_ 52.00     1       2.99/2.00/2.41   108      7.68    1.00  0.02    5630.25     0.00    /--     ---       ---%/---    % ---
  80607-0   72384    xjcorbalan python_executor_ 52.00     1       3.00/2.00/2.41   108      12.24   1.00  0.02    5632.75     0.00    /--     ---       ---%/---    % ---
  80607-0   72388    xjcorbalan python_executor_ 52.00     1       3.00/2.00/2.41   108      8.66    1.00  0.02    5630.46     0.00    /--     ---       ---%/---    % ---
  80607-0   72387    xjcorbalan python_executor_ 52.00     1       3.00/2.00/2.41   108      7.99    1.00  0.02    5632.65     0.00    /--     ---       ---%/---    % ---
  80607-0   72386    xjcorbalan python_executor_ 52.00     1       3.00/2.00/2.41   108      8.77    1.00  0.02    5632.28     0.00    /--     ---       ---%/---    % ---
  80607-0   72389    xjcorbalan python_executor_ 52.00     1       3.00/2.00/2.41   108      7.46    1.00  0.02
```

```
ear-job-visualizer -j 80607 --format ear2prv -o ~/paraver_traces/kmeans
```



Multiple executors in same node using EAR node sharing feature

Thanks!

Julita Corbalan julita.corbalan@as4dc.com, julita.corbalan@bsc.es

Benjamin Czaja (benjamin.czaja@surf.nl)